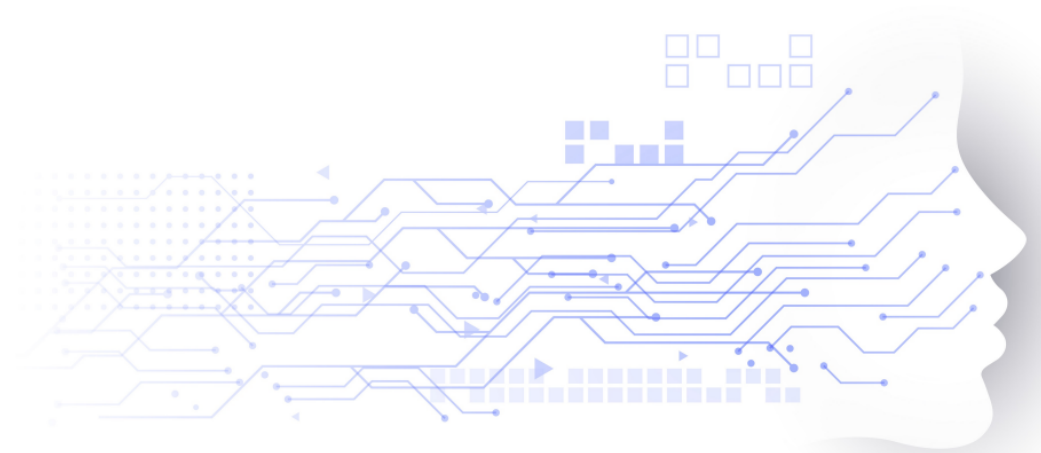


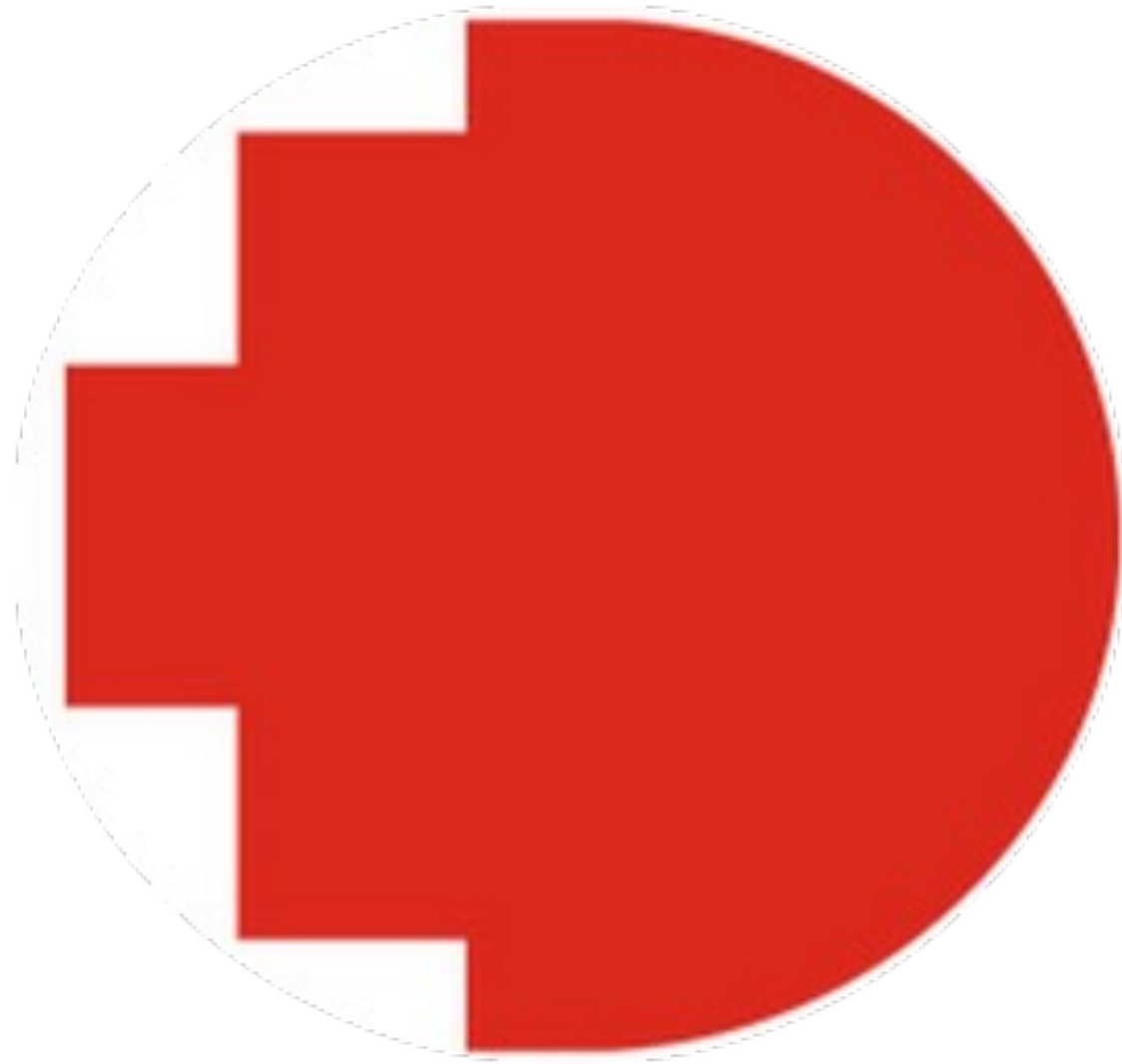


Regression

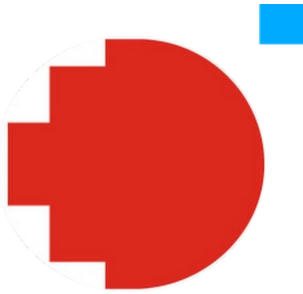
COSC2753 Machine Learning

Semester 2, 2026

Recap



What is Machine Learning?



Machine learning is programming computers to optimise a **performance** criterion/perform a particular **task** by **generalising** from examples of **past experience(s)** to **predict** what is to occur in future experience(s).

More technically

"A computer program is said to learn:

- *Some class of **tasks** T*
- *From **experience** E , and*
- ***Performance measure** P*

If its performance at tasks in T , as measured by P , improves with experience E "

- Tom Mitchell (1998)



Agenda



Regression

- Univariate linear regression
- Multivariate linear regression
- Practical issues

By the end of the lecture, you will:

- Understand the main concepts of regression including hypothesis spaces, cost functions and optimization.

Regression

Definition and examples

Regression



A form of supervised learning

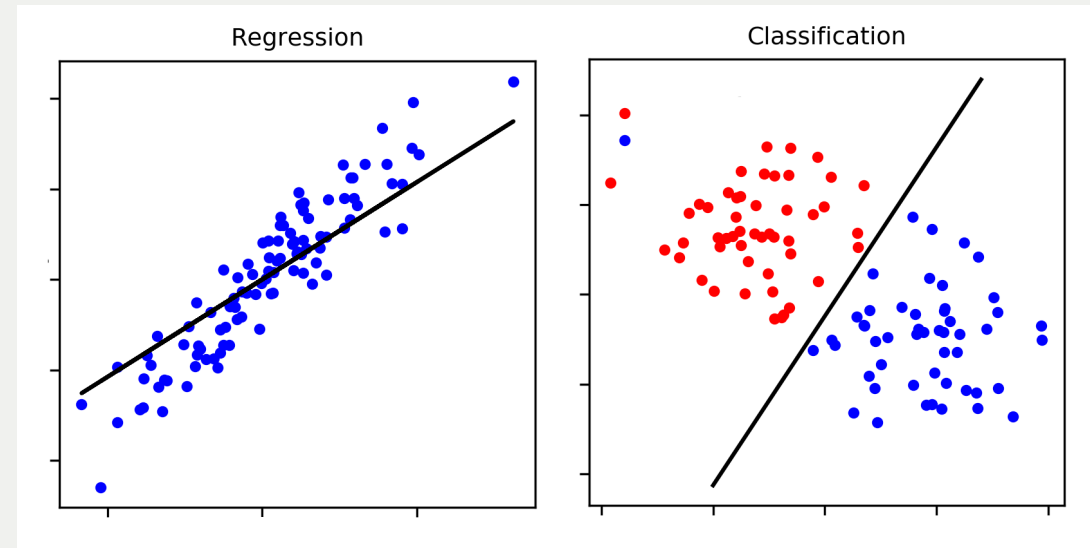
$$y = f(\mathbf{x})$$

The **output is continuous**:

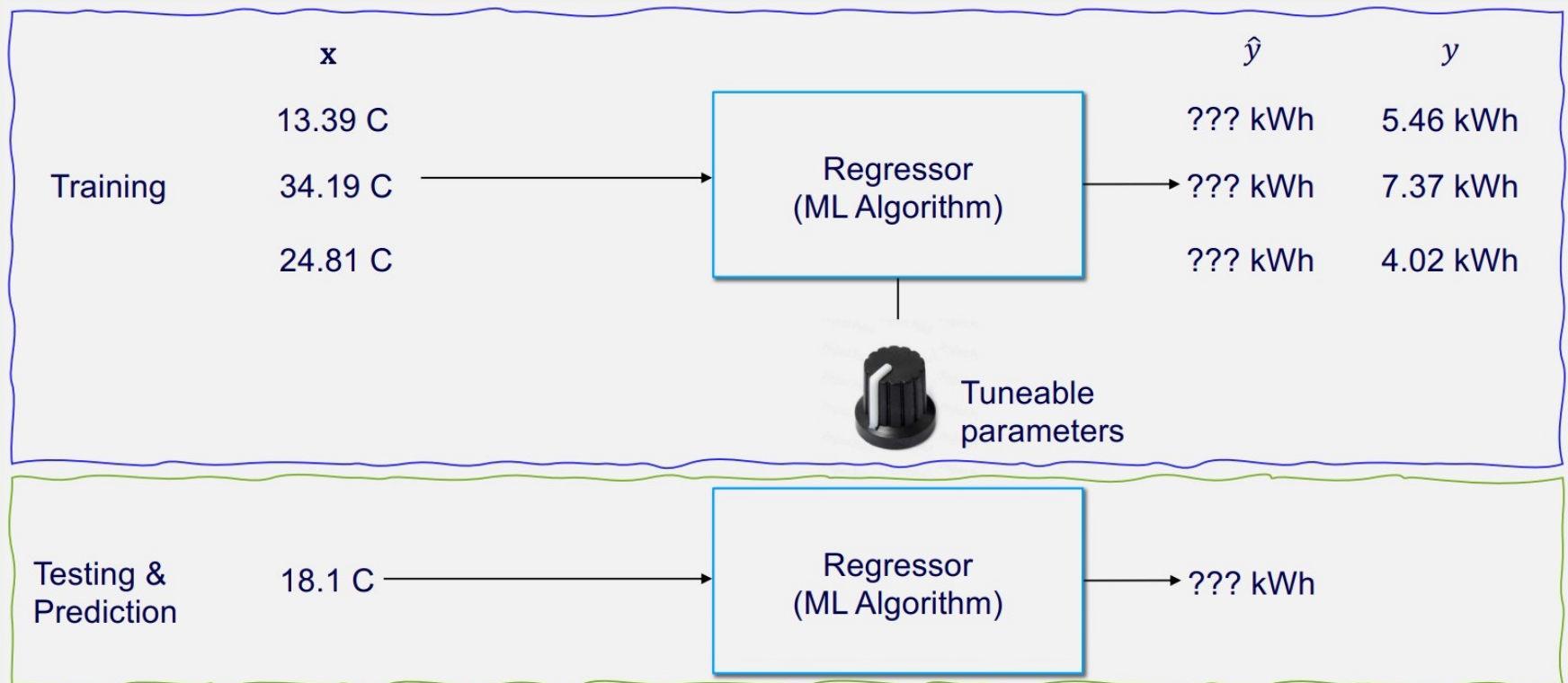
- $y \in \mathbb{R}$

Examples:

- Stock price prediction from google, amazon, Microsoft etc.
- Predicting hospital length-of-stay at time of admission.



Example: Power consumption



Experience



The experience is a data set (instances) of inputs and outputs:

$$D = \{x^{(i)}, f(x^{(i)})\}$$

Attributes of the task: x

- real-value, continuous
- independent variables

Outputs: y , **real-value**, continuous

- Dependent variable

Data representation



$$D = \{\mathbf{x}^{(i)}, f(\mathbf{x}^{(i)})\}$$

Univariate



Instance	Temperature (C)	Energy Consumption (kWh)
1	13.39	5.46
2	34.19	7.37
3	24.81	4.02

Multivariate



Instance	Temp (C)	Humidity	Pressure (ha)	Consumption (kWh)
1	13.39	0.81	1231	5.46
2	34.19	0.23	1638	7.37
3	24.81	0.45	1348	4.02

From last week



Nearly all ML algorithms can be described with a fairly simple recipe:

- Dataset (experience)
- Model (hypothesis space)
- Performance/Cost function (objective, loss)
- Optimization procedure

Let's gradually figure out what each element in the above recipe represent for regression.

Hypothesis space: linear regression

The simplest type of hypothesis space
(from many possible hypothesis spaces)

Linear regression



In **linear regression**, the Hypothesis is a **linear**** equation:

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- Features/attributes: $\mathbf{x} = [x_1, x_2, \dots, x_n]$
- Weights: $\theta = [\theta_0, \theta_1, \dots, \theta_n]$
- Hypothesis h , with respect to weights θ

*“Representing the hypothesis as a linear function (of attributes) is a **design choice** made by the ML engineer.”*

**No higher order terms in the hypothesis (e.g. x_1^2)

Simple univariate linear regression



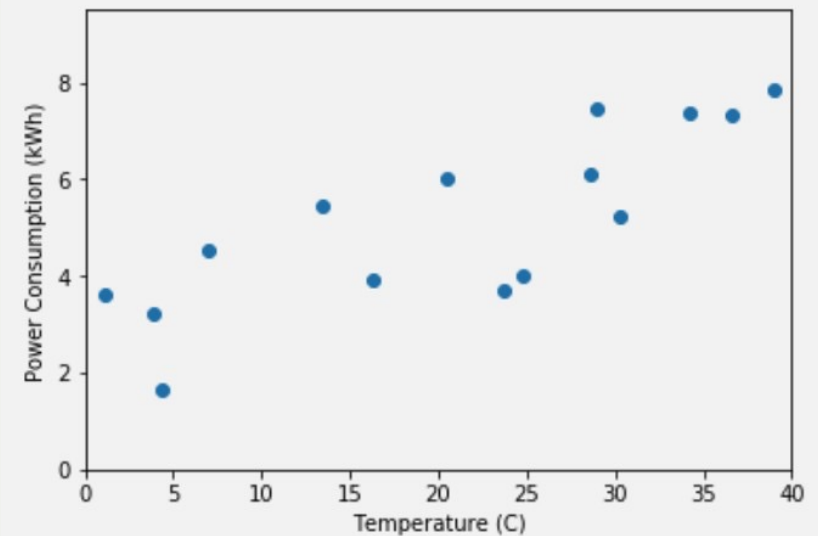
To discuss, consider univariate linear regression:

$$h_{\theta}(x): y = \theta_0 + \theta_1 x_1$$

- Intercept: θ_0
- Gradient: θ_1

This is a **parametric model**. The weights (or parameters) completely define the hypothesis (or model)

Instance	Temperature (C)	Energy Consumption (kWh)
1	13.39	5.46
2	34.19	7.37
3	24.81	4.02



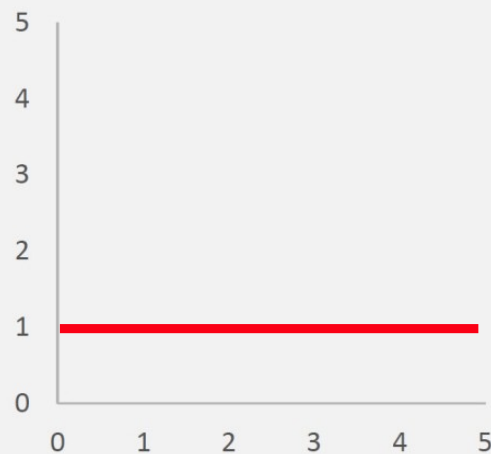
Simple univariate linear regression



Let's understand $h(x)$ and the effect of the parameters first

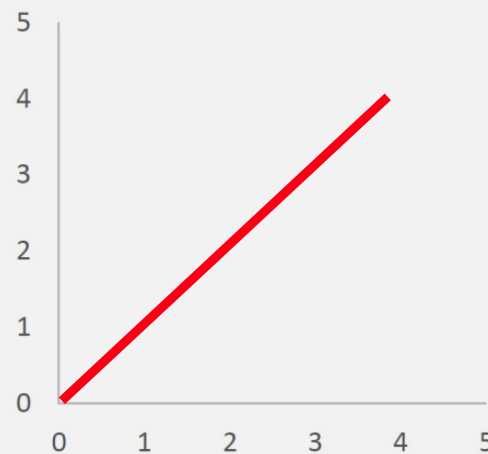
$$h_{\theta}(x): y = \theta_0 + \theta_1 x_1$$

$$h(x) = 1$$



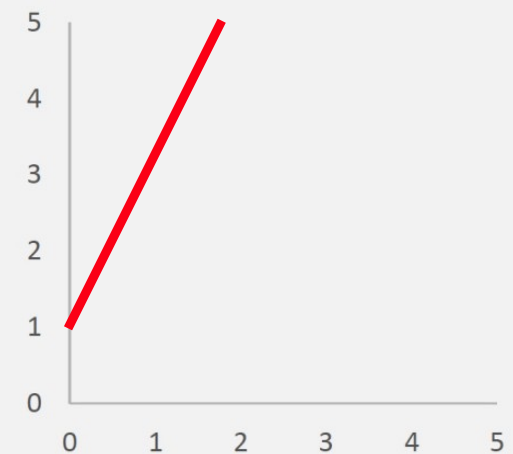
$$\begin{aligned}\theta_0 &= 1 \\ \theta_1 &= 0\end{aligned}$$

$$h(x) = x$$



$$\begin{aligned}\theta_0 &= 0 \\ \theta_1 &= 1\end{aligned}$$

$$h(x) = 2x + 1$$



$$\begin{aligned}\theta_0 &= 1 \\ \theta_1 &= 2\end{aligned}$$

Revision



- What is the purpose of including θ_0 ?
- For univariate case, the hypothesis can be visualized using a line. What geometric shape does the hypothesis take if we have two attributes?
- A particular problem has 5 input attributes. How many weights does the corresponding linear regression hypothesis have?

From last week



Nearly all ML algorithms can be described with a fairly simple recipe:

- Dataset (experience)
- Model (hypothesis space)
- Performance/Cost function (objective, loss)
- Optimization procedure

Let's gradually figure out what each element in the above recipe represent for regression.

Performance measure: linear regression

Quantifying the “goodness” of a hypothesis

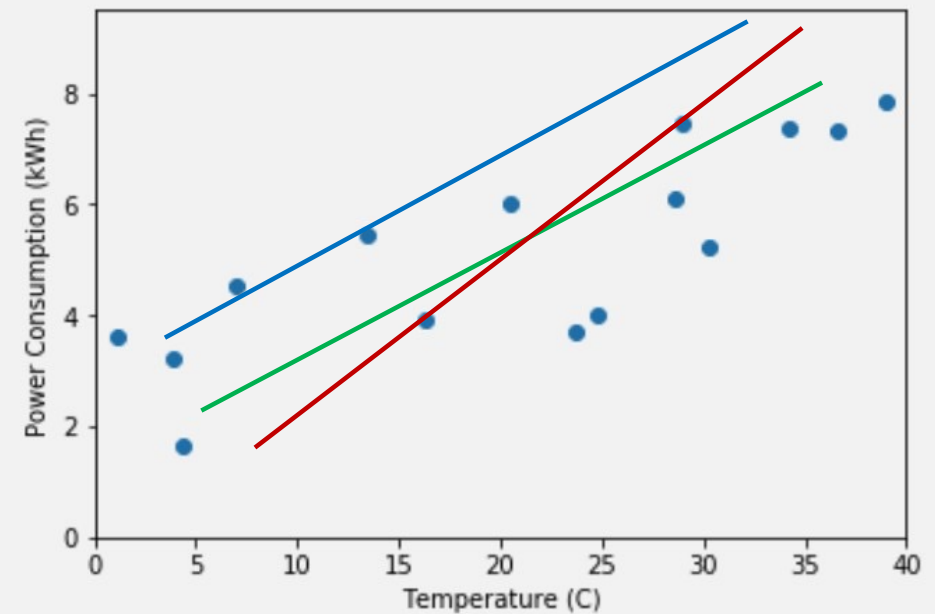
Linear regression goal



- Learning goal is to find the “**best**” regression line
- Need a measure of **performance**
- That is, the “line of best fit”
- How????

$$h_{\theta}(x): y = \theta_0 + \theta_1 x_1$$

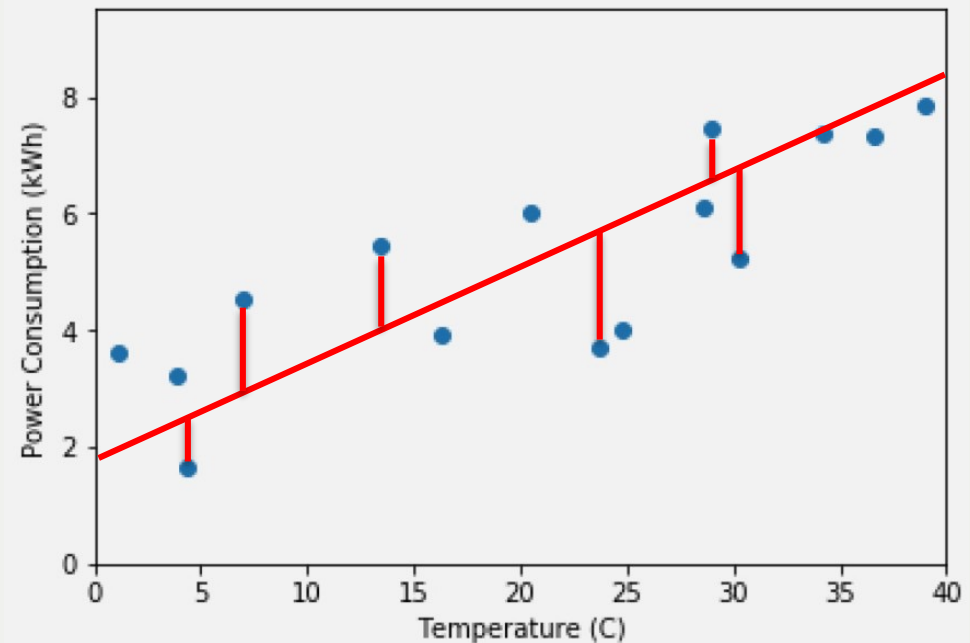
Instance	Temperature (C)	Energy Consumption (kWh)
1	13.39	5.46
2	34.19	7.37
3	24.81	4.02



Linear regression goal



- Learning goal is to find the “**best**” regression line
- That is, the “**line of best fit**”
- Minimize the **total distance** (error) between the hypothesis (regression line) and the actual training examples (x, y)



$$h_{\theta}(x): y = \theta_0 + \theta_1 x_1$$

Loss function

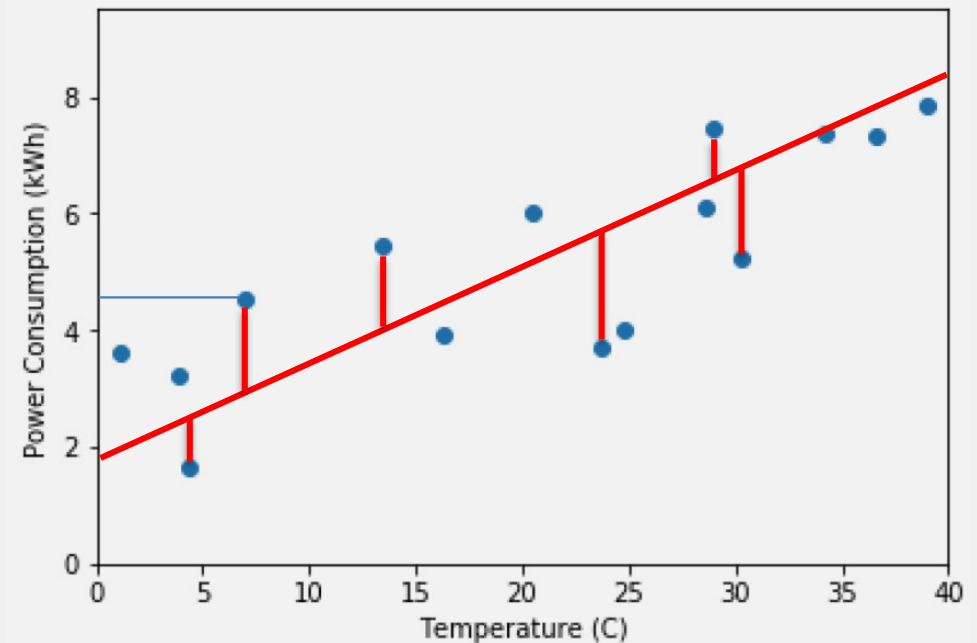


The measure of performance (**Loss Function**)

$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

Find a hypothesis that minimizes the sum of squared differences between the:

- Predicted output: $h_{\theta}(\mathbf{x}^{(i)})$
- Actual output: $y^{(i)}$
- For each training example: $\{\mathbf{x}^{(i)}, y^{(i)}\}$



$$h_{\theta}(\mathbf{x}): y = \theta_0 + \theta_1 x_1$$

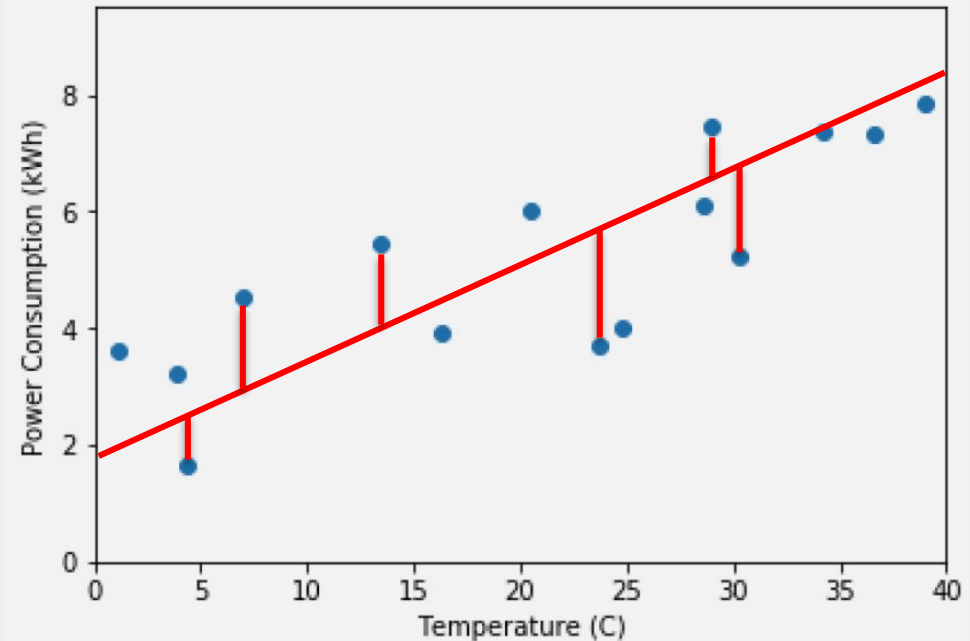
Loss function

Loss function is the measure of performance:

$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

Training goal: find θ_0 and θ_1 that minimizes $J(\theta_0, \theta_1)$

$$\theta_0^*, \theta_1^* = \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$



$$h_{\theta}(\mathbf{x}): y = \theta_0 + \theta_1 x_1$$

Simplified loss function intuition



What exactly does the goal mean? How does it relate to finding the best hypothesis?

First consider a simplified version of loss function, forcing $\theta_0 = 0$

Thus, the hypothesis is effectively: $h_{\theta}(\mathbf{x}) = \theta_1 x_1$

We only need to minimize $J(\theta_1)$, that is:

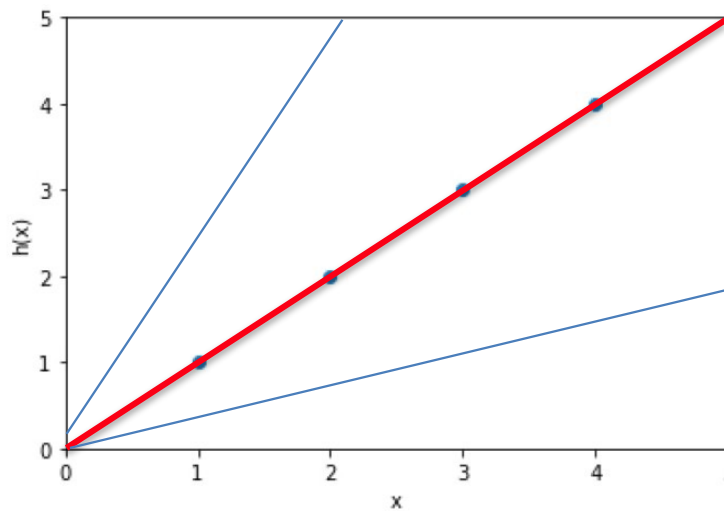
$$J(\theta_1) = \frac{1}{n} \sum_{i=1}^n (\theta_1 x_1^{(i)} - y^{(i)})^2$$

Goal:

$$\theta_1^* = \min_{\theta_1} J(\theta_1)$$

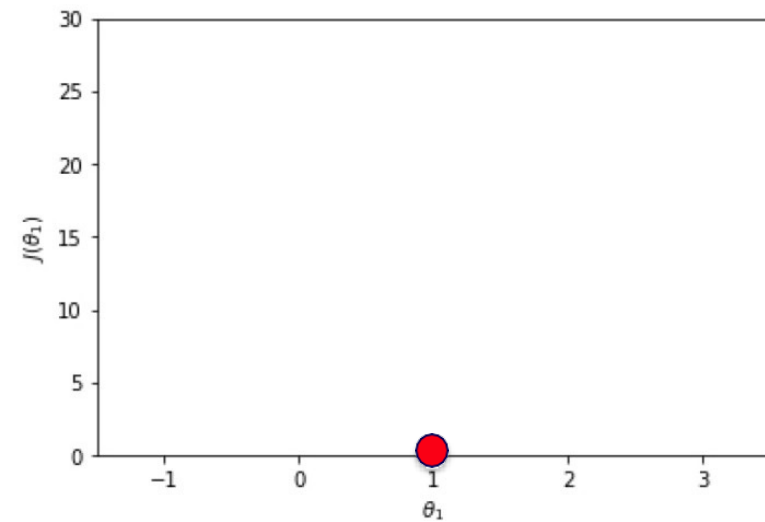
Simplified loss function intuition

$$h(x) = \theta_1 x$$



$$h(\mathbf{x}) = \theta_1 x \rightarrow h = x$$
$$\theta_1 = 1, n = 4$$

$$J(\theta_1) = \frac{1}{n} \sum_{i=1}^n (\theta_1 (x^{(i)}) - y^{(i)})^2$$

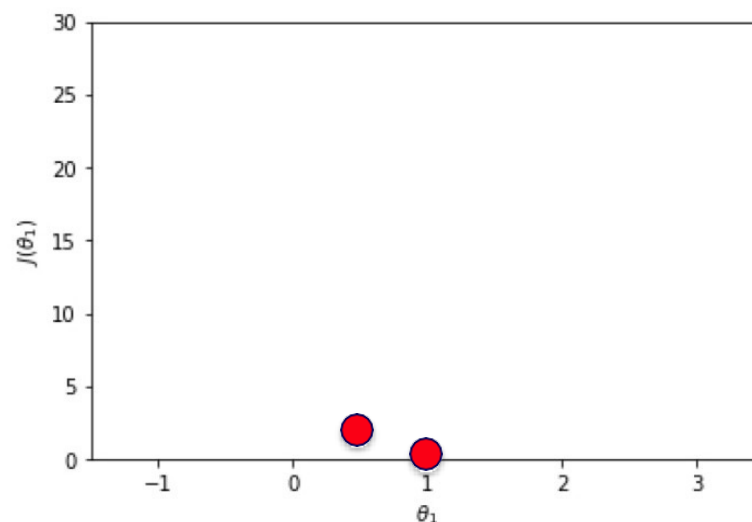
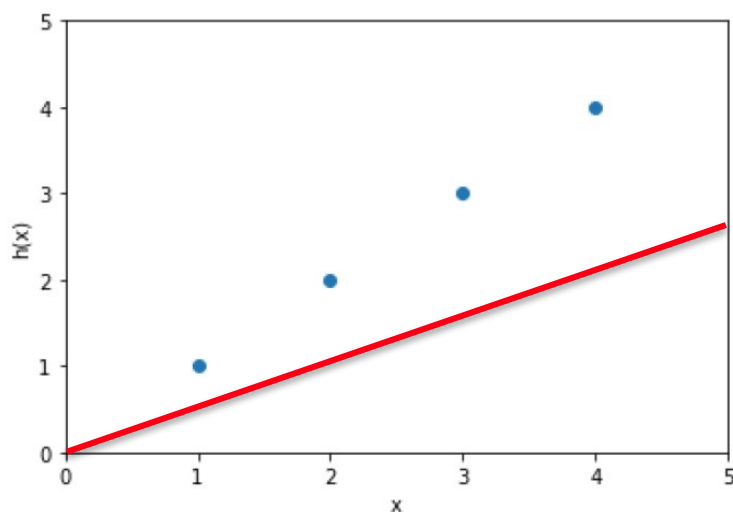


$$J(\theta_1) = \frac{1}{4} (0 + 0 + 0 + 0) = 0$$

Simplified loss function intuition

$$h(x) = \theta_1 x$$

$$J(\theta_1) = \frac{1}{n} \sum_{i=1}^n (\theta_1 (x^{(i)}) - y^{(i)})^2$$

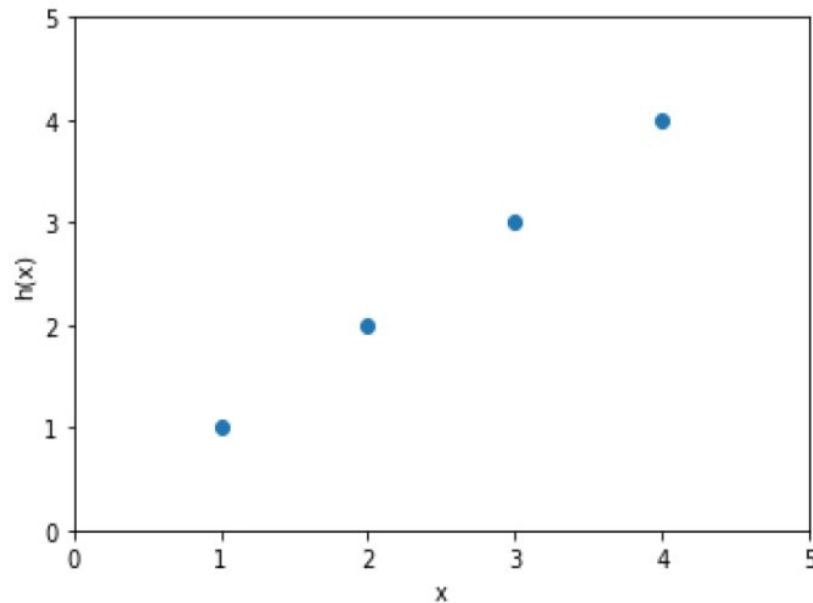


$$\begin{aligned} h(x) &= \theta_1 x \\ \theta_1 &= \frac{1}{2}, n = 4 \end{aligned} \rightarrow h = \frac{1}{2} x$$

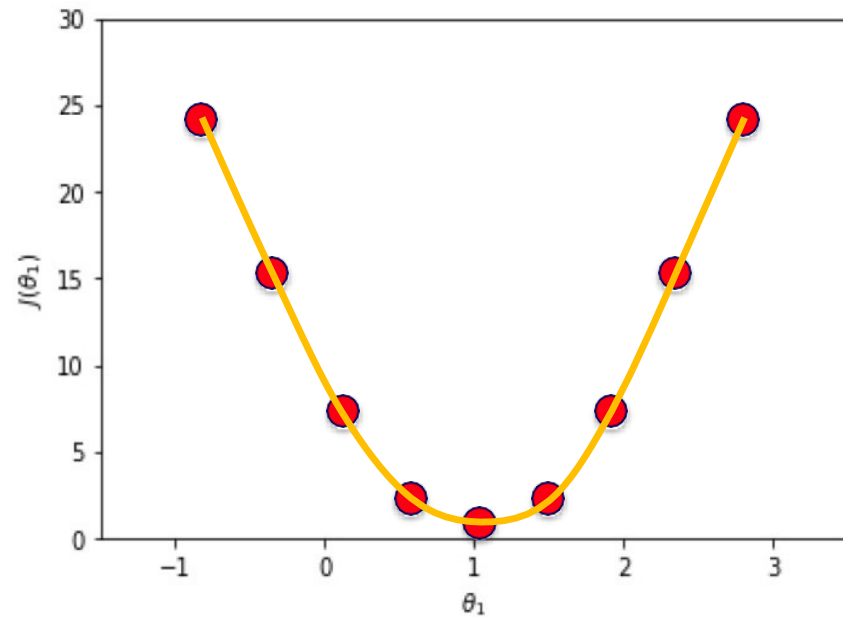
$$J(\theta_1) = \frac{1}{4} \left[\left(\frac{1}{2} - 1 \right)^2 + (1 - 2)^2 + \left(\frac{3}{2} - 3 \right)^2 + (2 - 4)^2 \right] = \frac{7.5}{4}$$

Simplified loss function intuition

$$h(x) = \theta_1 x$$



$$J(\theta_1) = \frac{1}{n} \sum_{i=1}^n (\theta_1 (x^{(i)}) - y^{(i)})^2$$

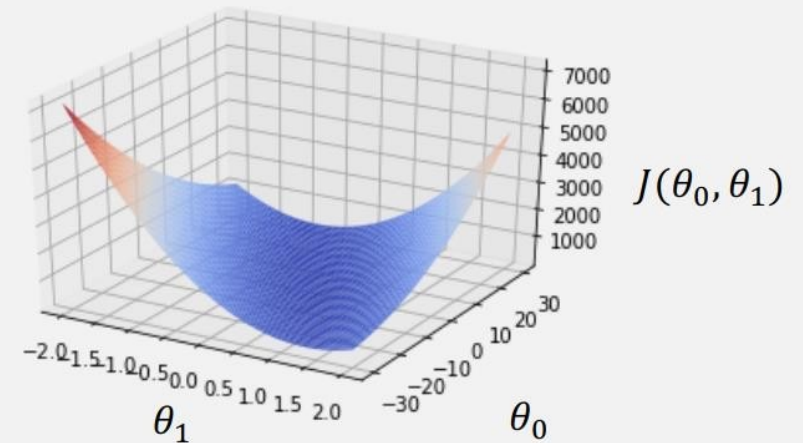
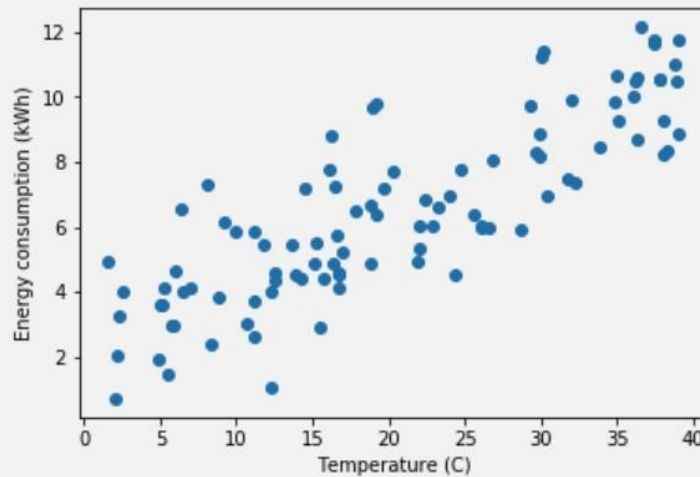


Complete loss function intuition



$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1$$

$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$$



Complete loss function intuition



- Hypothesis: $h_{\theta}(\mathbf{x}) = \theta_1 x + \theta_0$
- Loss function: $J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$
- Parameters: θ_0, θ_1
- Goal: $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$

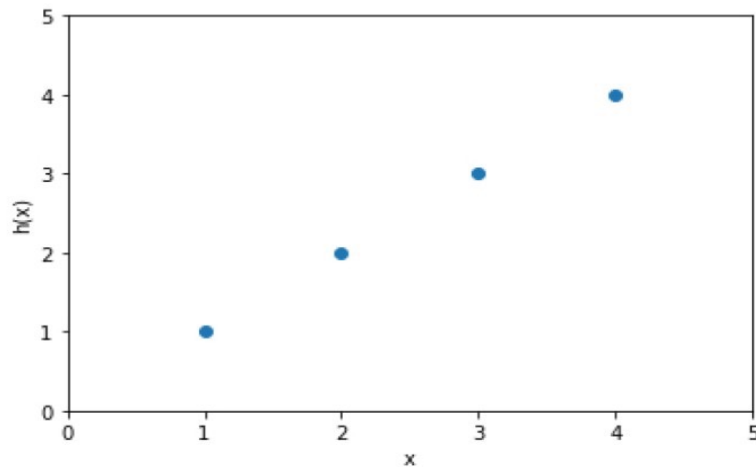
Optimization procedure: Gradient descent

Finding the optimal hypothesis automatically & efficiently

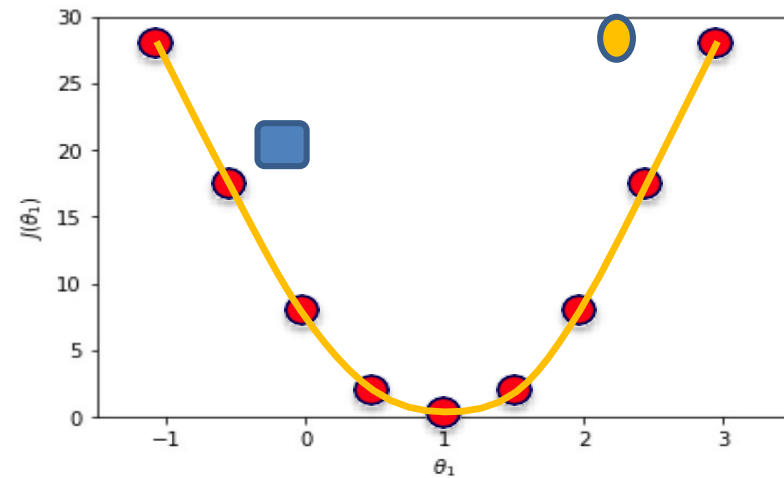
Minimizing the loss function

Reconsider simplified problem

$$h(x) = \theta_1 x$$



$$J(\theta_1) = \frac{1}{n} \sum_{i=1}^n (\theta_1 (x^{(i)}) - y^{(i)})^2$$

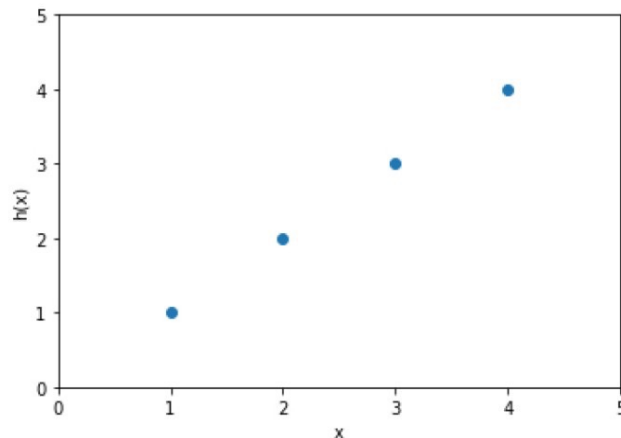


How can we search for the parameter(s), that minimize the loss function????

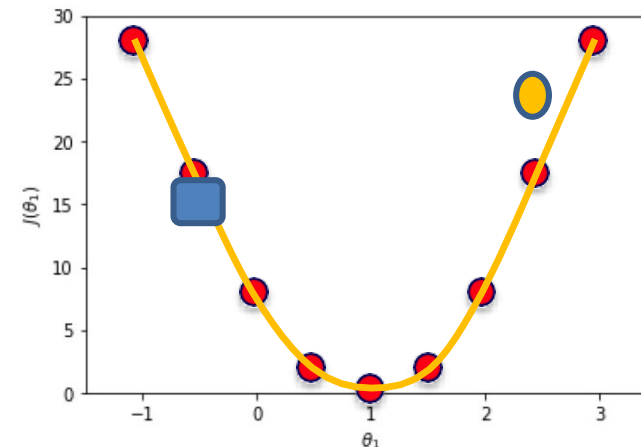
Minimizing the loss function

Reconsider simplified problem

$$h(x) = \theta_1 x$$



$$J(\theta_1) = \frac{1}{n} \sum_{i=1}^n (\theta_1 (x^{(i)}) - y^{(i)})^2$$



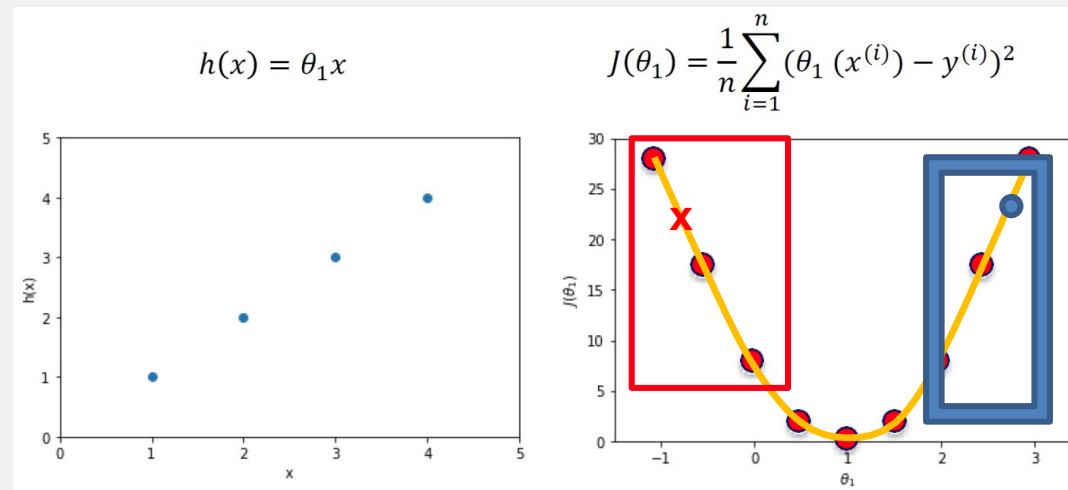
How can we search for the parameter(s), that minimize the loss function?

- We can do random search: Pick random parameter values and evaluate loss function. Choose the parameter with the minimum cost as the optimal.
- **Not a good idea:** we don't know where to search. How many evaluations to do?

Minimizing the loss function



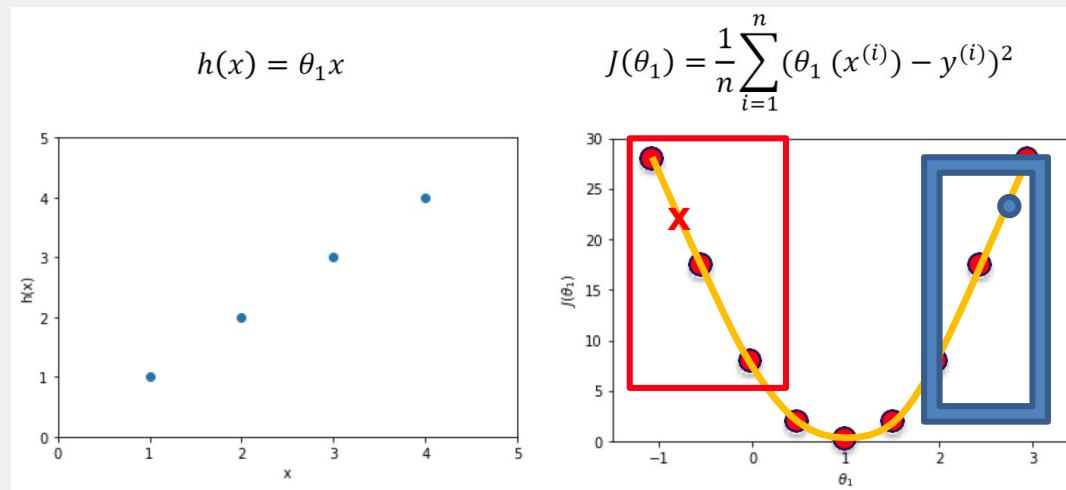
Reconsider simplified problem



Question: if I started at the red, **X** point on the above loss function, **should I increase or decrease θ_1** to find minimum (where is the minimum)?

Minimizing the loss function

Using gradients



Question 1: if I started at the red, **X** point on the above loss function, should I increase or decrease θ_1 to find minimum (where is the minimum)?

→ use the information in the “slope” (**gradient**) to do the search cleverly.

Using gradients



Recall we want to find minimize the loss function,

$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

General approach:

- Start with some θ_0, θ_1

- Could be random

- Could be based on some heuristics or rules or other ML approach

- Update θ_0, θ_1 such that it reduces $J(\theta_0, \theta_1)$

- Use gradients, that is the derivative of $J(\theta_0, \theta_1)$

- Repeat until the minimum is found

- Hopefully the global minimum

Differentiating the loss function



Require the derivative of the loss function:

$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

Derivative with respect to what variable(s)?

For minimizing loss function, this is the weights: θ_0, θ_1

As the attributes are **independent**, so are the weights → can take partial derivatives!

Partial derivatives (analytical)



$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

$$\begin{aligned} \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_j} &= \frac{1}{n} \sum_{i=1}^n \frac{\partial}{\partial \theta_j} (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 = \frac{1}{n} \sum_{i=1}^n 2(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \frac{\partial}{\partial \theta_j} (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \\ &= 2 \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \left[\frac{\partial}{\partial \theta_j} h_{\theta}(\mathbf{x}^{(i)}) - \frac{\partial}{\partial \theta_j} y^{(i)} \right] = 2 \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \left[\frac{\partial}{\partial \theta_j} h_{\theta}(\mathbf{x}^{(i)}) - 0 \right] \\ &= 2 \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \frac{\partial}{\partial \theta_j} h_{\theta}(\mathbf{x}^{(i)}) \end{aligned}$$

$$\frac{\partial}{\partial \theta_0} h_{\theta}(\mathbf{x}^{(i)}) = \frac{\partial}{\partial \theta_0} (\theta_0 + \theta_1 \mathbf{x}^{(i)}) = \frac{\partial}{\partial \theta_0} \theta_0 + \frac{\partial}{\partial \theta_0} \theta_1 \mathbf{x}^{(i)} = 1 + 0 = 1$$

$$\frac{\partial}{\partial \theta_1} h_{\theta}(\mathbf{x}^{(i)}) = \frac{\partial}{\partial \theta_1} (\theta_0 + \theta_1 \mathbf{x}^{(i)}) = \frac{\partial}{\partial \theta_1} \theta_0 + \frac{\partial}{\partial \theta_1} \theta_1 \mathbf{x}^{(i)} = 0 + \mathbf{x}^{(i)} = \mathbf{x}^{(i)}$$

How to calculate derivatives?

What is a partial derivative?

What is the chain rule?

How calculate partial derivatives?

Give it a go!

If need, pls let us know

Gradient descent – Univariate linear regression



1. Initialize θ_0 and θ_1 (can be randomly selected)
2. Until converged:
 1. Compute gradients: $\nabla_{\theta} J(\theta_0, \theta_1) = \left[\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0}, \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} \right]$
 2. Update parameters (weights):
$$\theta_0 \leftarrow \theta_0 - a \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0}$$
$$\theta_1 \leftarrow \theta_1 - a \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1}$$

Step size (learning rate): a

How can we determine when the algorithm has converged?

- When the change in loss function due the **update is small**
- Pre-determined **number of iterations**

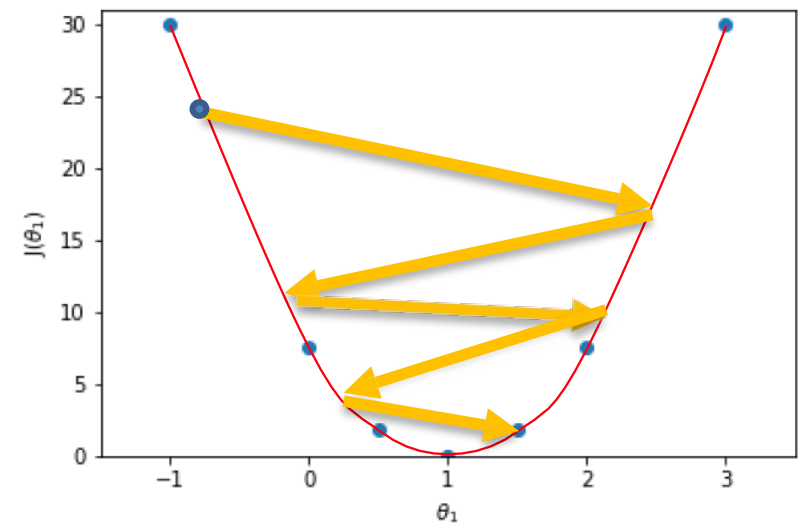
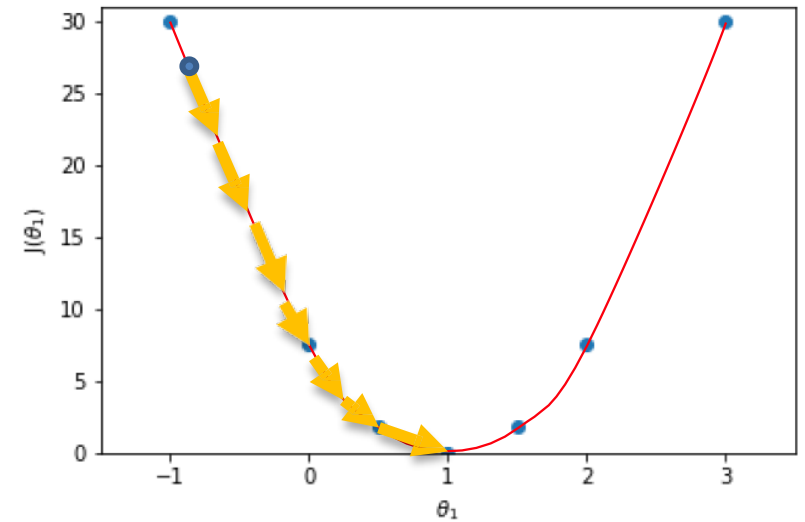
Selecting step size

This is what we call a hyperparameter. A parameter that is part of the learning algorithm (not the model or hypothesis).

- If step size is set **too low**, it will take a long time to get to the optimal.
- If the step size is set **too large**, it will overshoot the optimal and oscillate without converging.

How to set?

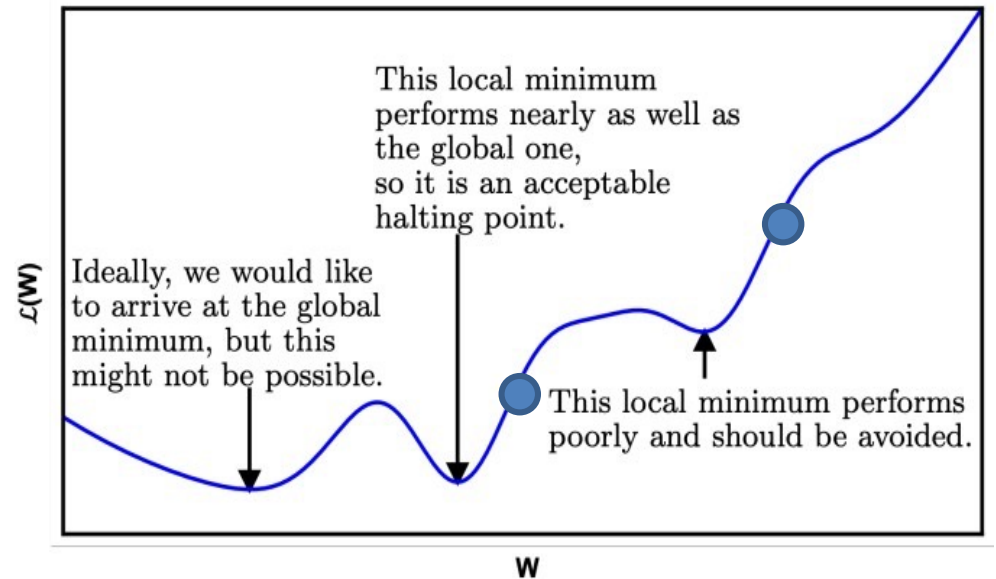
Trial and error, prior experience or guidelines. There are also adaptive methods that can determine the step size value automatically.



Global vs. Local minima

Even with the appropriate learning rate, **we may not converge** to the best optimum

- In general, gradient descent will find a local optimum
- For linear regression we **WILL** find an optimal solution with gradient descent
- This is because the linear regression loss function is convex and has only one minimum (the global minimum)



Multi-variate regression

Regression models with multiple variables

Multiple features (variables)



Instance	Temp (Deg C)	No. of people	Size of house	Area of windows	Energy Consumption (kWh)
1	15	2	70	40	3400
2	8	3	160	78	5000
3	26	4	300	50	8000
...	...	...	...	...	...
n	35	1	250	100	7000

Multi-variate hypothesis



Univariate hypothesis: $h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x$

Multivariate hypothesis:

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_m x_m$$

For m attributes (n is typically used for the size of the data set)

Gradient descent



Hypothesis: $h_{\theta}(\mathbf{x}) = \theta^T \mathbf{x} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m$

Parameters: $\theta_0, \theta_1, \dots, \theta_m$

Loss function: $J(\theta_0, \theta_1, \dots, \theta_m) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$

Gradient descent algorithm:

Repeat {

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1, \dots, \theta_m)$$

}

Issues

Some practical issues with regression models

Issues: Hypothesis



We can ask the question about the nature of the hypothesis:

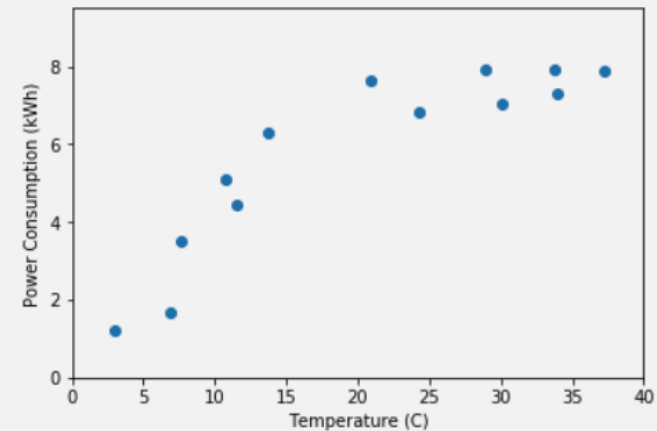
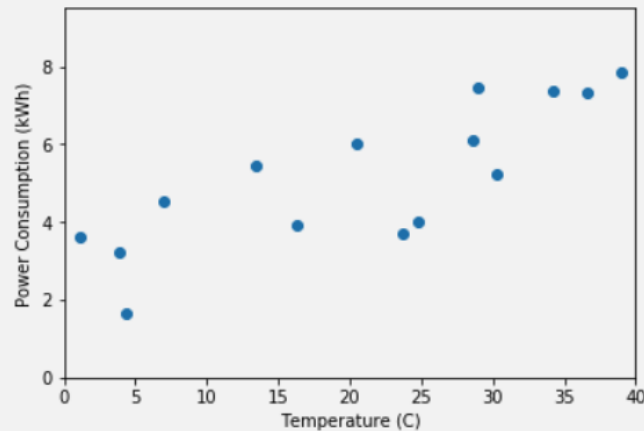
Is the regression model (choice of weights) good?

That is, the hypothesis $h(\mathbf{x})$ is a good choice from the Hypothesis Space H ?

Is linear regression suitable?

That is, does the hypothesis approximate the unknown target function?

$$h(\mathbf{x}) \approx f(\mathbf{x})$$



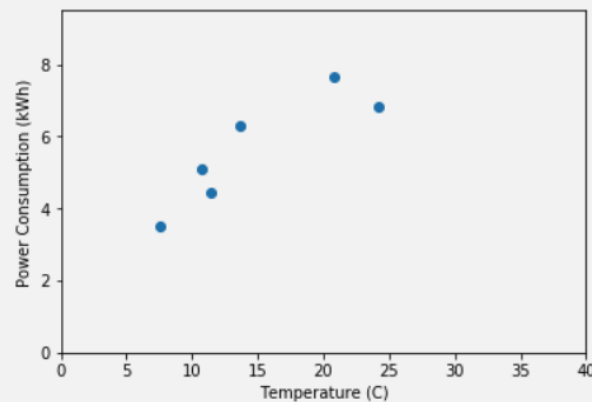
Issues: Prediction



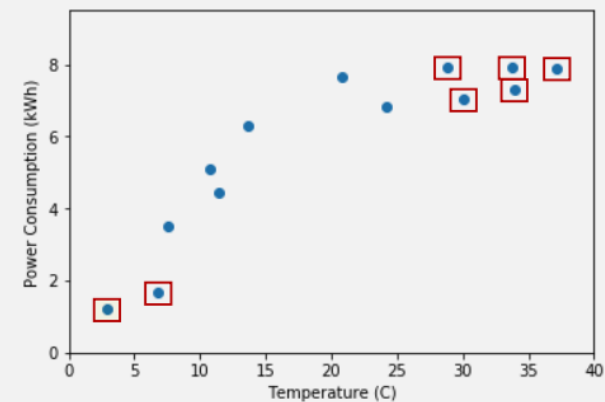
We can also ask questions about what the hypothesis predicts:

Is a prediction for an unseen example any good?

That is, how close are $h(\mathbf{x}_{unknown})$ and $f(\mathbf{x}_{unknown})$?



Train data



Train data + unseen data

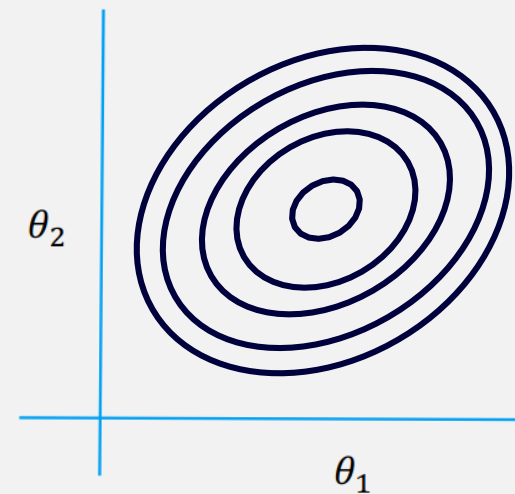
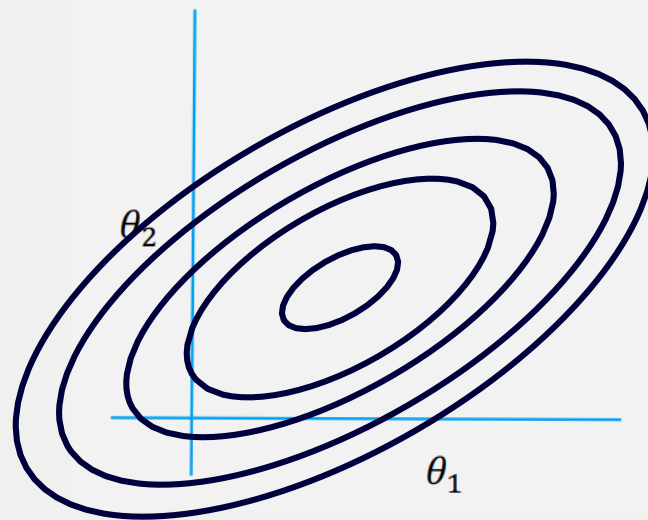
Practical issue: Feature scaling



What happens if two features (attributes) have very different domains?

x_1 = temperature (0 to 40 degrees Celsius)

x_2 = number of people (1 to 12 people)



Practical issue: Feature scaling



Regression works, but is slower

Prefer to keep **scale** all feature to the same proportion

Typically use **normalization**

Any reason why we need normalization?

Practical issue: Feature scaling



Replace each feature x_i with $x_i - \mu_i$ to make features to have approximately zero mean

Do not apply to $x_0 = 1$ (with θ_0)

e.g.,

$$x_1 = \frac{\text{temperature} - 20}{40}, -0.5 \leq x_1 \leq 0.5$$

$$x_2 = \frac{\text{number of people} - 6}{12}, -0.5 \leq x_2 \leq 0.5$$

Further adjust by the standard deviation i.e., $x_i = \frac{x_i - \mu_i}{\sigma_i}$

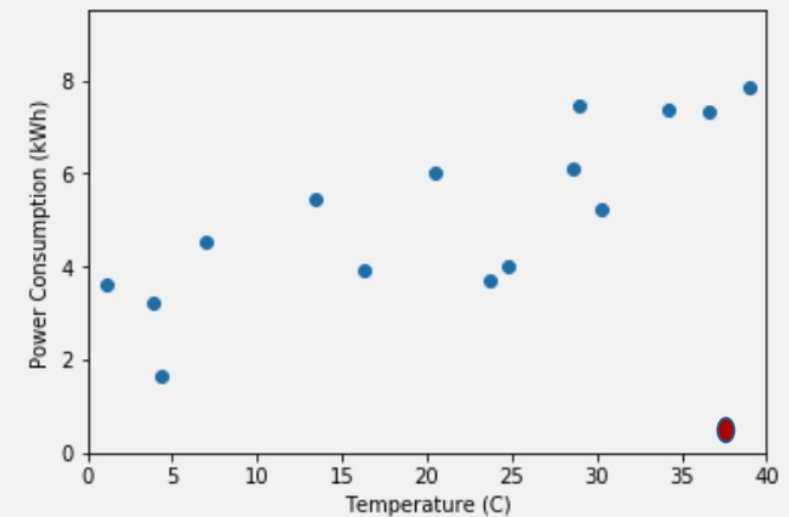
Issue: Data & Loss function



What happens if there are data points that does not agree with the other data points?

How can we overcome this?

- Clean the data
- Use robust cost functions – outside the scope of this course.



Summary



- Regression
- Uni-variate/Multi-variables Linear Regression
- Loss function
- Optimization
 - Gradient descent
 - Parameters fitting (intuition-learning rate)
- Practical issues

Thank you!